```
            std::thread::id old_id;
            if(hazard_pointers[i].id.compare_exchange_strong(
                old_id,std::this_thread::get_id()))
            {
                hp=&hazard_pointers[i];
                break;
            }
        }
        if(!hp)     <-- 1
        {
            throw std::runtime_error("No hazard pointers available");
        }
    }

    std::atomic<void*>& get_pointer()
    {
        return hp->pointer;
    }

    ~hp_owner()     <-- 2
    {
        hp->pointer.store(nullptr);
        hp->id.store(std::thread::id());
    }
};

std::atomic<void*>& get_hazard_pointer_for_current_thread()     <-- 3
{
    thread_local static hp_owner hazard;     <-- 4
    return hazard.get_pointer();     <-- 5
}
```

Try to claim ownership of a hazard pointer

Each thread has its own hazard pointer

The actual implementation of `get_hazard_pointer_for_current_thread()` itself is deceptively simple ❸: it has a `thread_local` variable of type `hp_owner` ❹ that stores the hazard pointer for the current thread. It then just returns the pointer from that object ❺. This works as follows: The first time *each thread* calls this function, a new instance of `hp_owner` is created. The constructor for this new instance ❶ then searches through the table of owner/pointer pairs looking for an entry without an owner. It uses `compare_exchange_strong()` to check for an entry without an owner and claim it in one go ❷. If the `compare_exchange_strong()` fails, another thread owns that entry, so you move on to the next. If the exchange succeeds, you've successfully claimed the entry for the current thread, so you store it and stop the search